A

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Midterm Exam | Semester: Fall 2025
Duration: 1 Hour 20 minutes | Full Marks: 25

CSE 221: Algorithms

Answer the following questions.
Figures in the right margin indicate marks.

Name:	ID:	Section:

1 a. CO2 Why do we ignore constants and lower-order terms when expressing the time complexity of an algorithm? Briefly **explain** in your own words. **01**

b. CO2 Is $n * \sqrt{n}$ asymptotically faster than $n * log_2(n)$? **Justify** with proper logic. **02**

c. CO2 **Write** the time complexity of the following code snippet. **03**

```
public static void func(int[] arr) {
    int n = arr.length;
    int k = 0;
    for (int i = 0; i < n; i++)
        k += arr[i];

    for (int i = 1; i < n; i++)
        for (int j = 1; j <= arr[i]; j++)
            System.out.print(i);
}
```

2 a. CO1 You are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A: \{15, -13, 3, -4, 5, -6\}$. **05**

You have to organize the array in a way that all the negative numbers come to the front and all positive numbers are pushed to the back. For example, the array A from above becomes $Org_A: \{-6, -4, -13, 15, 3, 5\}$. *Note that numbers don't need to be sorted here.*

Propose a linear-time $[O(N)]$ algorithm with constant $[O(1)]$ space complexity. **Present** your algorithm as pseudocode or executable code, or step-by-step logical instructions.

If unable, show a simulation of any algorithm (no limit on the time or space complexity) on the given example array A for 40% marks.

b. CO1 After solving Q2(a), you want to find the index of the leftmost positive number in the organized array Org_A. Present a modification of Binary Search to achieve this. **Write** either pseudocode or executable code. **03**

c. CO2 Mention the name and time complexity of the most efficient sorting algorithm among Bubble, Insertion, Selection, Merge, Quick, and Count for each of the following input scenarios, consisting of an array of N integers. For each case, you do not have any information other than the given one. **02**

Case #1: The largest number can be in any index, but all other numbers are already sorted.
Case #2: The smallest number is 0, and the largest is 2^N.
Case #3: The array contains all the integers from 1 to N.
Case #4: The numbers are in the reverse order of how we want them to be sorted.

3 We are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A: \{15, \ -13, 3, \ -4, 5, \ -6\}$.

We want to find the **Max-Prod Subarray** of the input. Just like the Max-sum Subarray, but instead of summation, we have to consider the product (multiplication) of the numbers.

We have come up with a divide-and-conquer algorithm to solve this problem. This algorithm has the same concept that you know for the Max-sum Subarray, just modified for multiplication.

```
function MaxProdSubarray(A, left, right){
    if (left == right):
        return A[left]

    mid = (left + right) / 2

    LPS = MaxProdSubarray(A, left, mid)
    RPS = MaxProdSubarray(A, mid + 1, right)
    CPS = CrossProduct(A, left, mid, right)

    return max(LPS, RPS, CPS)
}

function CrossProduct(A, left, mid, right){
    leftProd = -∞
    prod = 1
    for (i from mid to left){
        prod = prod * A[i]
        if (prod > leftProd): leftProd = prod
    }

    rightProd = -∞
    prod = 1
    for (i from mid+1 to right){
        prod = prod * A[i]
        if (prod > rightProd): rightProd = prod
    }
    return leftProd * rightProd
}
```

a. **CO1** **Mark** the changes that you see in this algorithm than the one for Max-Sum Subarray. **02**
You can either write only those lines in your script. Or, write the whole code given above and underline the ones you want to mark. Also, you don't have to mark the changes in variable and function names.

b. **CO3** **Evaluate** whether the given algorithm would actually work for this problem. **03**
If your answer is yes, give a proper logical explanation. Otherwise, give an input where the algorithm will not produce the correct answer.

c. **CO2** We are thinking, why not use Karatsuba's multiplication technique to make our algorithm faster? So we replace the multiplication operations (*) with Karatsuba's function. For example, **02**
$prod * A[i]$ becomes $Karatsuba(prod, A[i])$.

Write a recurrence relation [$T(N) =$ ___ $+$ ___] to express the time complexity of the algorithm after applying Karatsuba. Assume that Karatsuba's time complexity is $O(N^{1.58})$ (same N as the size of the Array A).

d. **CO2** Now **solve** your recurrence from Q3(c) to find the actual time complexity. **02**
Any method is acceptable as long as you show proper work.

B

BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Midterm Exam | Semester: Fall 2025
Duration: 1 Hour 20 minutes | Full Marks: 25

CSE 221: Algorithms

Answer the following questions.
Figures in the right margin indicate marks.

Name:	ID:	Section:

1 a. CO2 Why do we ignore constants and lower-order terms when expressing the time complexity of an algorithm? Briefly **explain** in your own words. **01**

b. CO2 Is $\sqrt{n} * log_2(n)$ asymptotically faster than n? **Justify** with proper logic. **02**

c. CO2 **Write** the time complexity of the following code snippet. **03**

```
public static void func(int[] arr) {
    int n = arr.length;
    int k = 0;
    for (int i = 0; i < n; i++)
        k += arr[i];

    for (int i = 1; i < n; i++)
        for (int j = 1; j <= k; j++)
            System.out.print(i);
}
```

2 a. CO1 You are given an array B consisting of N integers. Some of the numbers are even, and some are odd. For example $B: \{7, 2, 9, 4, 11, 6\}$. **05**

You have to organize the array in a way that all the even numbers come to the front and all odd numbers are pushed to the back. For example, the array B from above becomes $Org_B: \{6, 2, 4, 9, 11, 7\}$. *Note that numbers don't need to be sorted here.*

Propose a linear-time $[O(N)]$ algorithm with constant $[O(1)]$ space complexity. **Present** your algorithm as pseudocode or executable code, or step-by-step logical instructions.

If unable, show a simulation of any algorithm (no limit on the time or space complexity) on the given example array B for 40% marks.

b. CO1 After solving Q2(a), you want to find the index of the rightmost even number in the organized array Org_B. Present a modification of Binary Search to achieve this. **Write** either pseudocode or executable code. **03**

c. CO2 Mention the name and time complexity of the most efficient sorting algorithm among Bubble, Insertion, Selection, Merge, Quick, and Count for each of the following input scenarios, consisting of an array of N integers. For each case, you do not have any information other than the given one. **02**

Case #1: The smallest number can be in any index, but all others are already sorted.
Case #2: The smallest number is 0, and the largest is 2^N.
Case #3: The array contains all the integers from 1 to N.
Case #4: Just some random numbers, no pattern could be seen.

3 We are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A: \{5, -3, 13, 4, -5, -6\}$.

We want to find the **Max-Prod Subarray** of the input. Just like the Max-sum Subarray, but instead of summation, we have to consider the product (multiplication) of the numbers.

We have come up with a divide-and-conquer algorithm to solve this problem. This algorithm has the same concept that you know for the Max-sum Subarray, just modified for multiplication.

```
function MaxProdSubarray(A, left, right){
    if (left == right):
        return A[left]

    mid = (left + right) / 2

    LPS = MaxProdSubarray(A, left, mid)
    RPS = MaxProdSubarray(A, mid + 1, right)
    CPS = CrossProduct(A, left, mid, right)

    return max(LPS, RPS, CPS)
}

function CrossProduct(A, left, mid, right){
    leftProd = -∞
    prod = 1
    for (i from mid to left){
        prod = prod * A[i]
        if (prod > leftProd): leftProd = prod
    }

    rightProd = -∞
    prod = 1
    for (i from mid+1 to right){
        prod = prod * A[i]
        if (prod > rightProd): rightProd = prod
    }
    return leftProd * rightProd
}
```

a. **CO1** **Mark** the changes that you see in this algorithm than the one for Max-Sum Subarray. You can either write only those lines in your script. Or, write the whole code given above and underline the ones you want to mark. Also, you don't have to mark the changes in variable and function names. **02**

b. **CO3** **Evaluate** whether the given algorithm would actually work for this problem. If your answer is yes, give a proper logical explanation. Otherwise, give an input where the algorithm will not produce the correct answer. **03**

c. **CO2** We are thinking, why not use Karatsuba's multiplication technique to make our algorithm faster? So we replace the multiplication operations (*) with Karatsuba's function. For example, **02**

$prod * A[i]$ becomes $Karatsuba(prod,\ A[i])$.

Write a recurrence relation [$T(N) =$ ___ $+$ ___] to express the time complexity of the algorithm after applying Karatsuba. Assume that Karatsuba's time complexity is $O(N^{1.58})$ (same N as the size of the Array A).

d. **CO2** Now **solve** your recurrence from Q3(c) to find the actual time complexity. Any method is acceptable as long as you show proper work. **02**